

NBA Decisioning Lab v1

API Documentation - Local Synthetic Next-Best-Action Training Service

Scope: This document describes the local API created for the NBA Decisioning Lab v1. It is not a Nordea API and does not connect to real bank systems.

1. Purpose

The NBA Decisioning Lab v1 API is a local training API for a synthetic Next-Best-Action (NBA) banking decision system. Version 1 is deliberately rule-based so you can clearly see the difference between customer data, eligibility, scoring, ranking, explainability, API delivery, and automated testing.

2. Technology

Component	Role
FastAPI	Defines HTTP endpoints and response models.
Uvicorn	Runs the FastAPI application as a local ASGI web server.
Pydantic	Validates API data models.
pandas	Loads the synthetic customer dataset.
pytest	Runs automated tests.

```
uvicorn src.app:app --reload
```

Meaning: `src.app` loads `src/app.py`; `:app` selects the FastAPI object named `app`; `--reload` restarts the server when source files change.

3. Base URL and Built-in Documentation

Resource	URL
Base URL	http://127.0.0.1:8000
Swagger UI	http://127.0.0.1:8000/docs
OpenAPI JSON	http://127.0.0.1:8000/openapi.json

4. High-Level Flow

```
Client / Browser / Postman
|
v
Uvicorn
|
v
FastAPI
|
v
Customer Repository
|
v
Decision Engine
|
v
Eligibility + Scoring + Ranking
|
v
Next Best Action Response
```

5. Endpoint - GET /health

Checks whether the local API is running.

GET /health

Response:

```
{
  "status": "ok",
  "version": "1.0.0"
}
```

Status: 200 OK when the service is available.

6. Endpoint - GET /nba/{customer_id}

Returns the Next Best Action for one synthetic customer.

Parameter	Type	Required	Description
customer_id	string	Yes	Synthetic customer ID, for example C00001.

GET /nba/C00001

PowerShell:

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/nba/C00001" -Method Get
```

cURL:

```
curl http://127.0.0.1:8000/nba/C00001
```

Example successful response

```
{
  "customer_id": "C00001",
  "next_best_action": "SAVINGS_PLAN",
  "score": 0.74,
  "eligible": true,
  "reason_codes": [
    "HAS_MONTHLY_SURPLUS",
    "LOW_SAVINGS_RELATIVE_TO_INCOME",
    "DIGITAL_ENGAGEMENT"
  ],
  "ranked_actions": [
    {
      "action": "SAVINGS_PLAN",
      "score": 0.74,
      "eligible": true,
      "reason_codes": [
        "HAS_MONTHLY_SURPLUS"
      ]
    }
  ]
}
```

The exact result depends on the synthetic customer's data and the current v1 rules.

Errors

404 Not Found

```
{
  "detail": "Customer not found"
}
```

7. Candidate Actions

Action	Meaning
SAVINGS_PLAN	Savings-related recommendation.
INVESTMENT_INFO	Investment-related information.

Action	Meaning
MORTGAGE_CONSULTATION	Mortgage discussion recommendation.
CREDIT_CARD_UPGRADE	Credit-card upgrade recommendation.
FINANCIAL_HEALTH_CHECK	Financial health review recommendation.

8. Response Schema

Field	Type	Description
customer_id	string	Synthetic customer identifier.
next_best_action	string	Winning action.
score	number	Final score from 0.0 to 1.0.
eligible	boolean	Whether the winning action is allowed.
reason_codes	array[string]	Explanation for the winning decision.
ranked_actions	array	All candidate actions after evaluation.

ActionScore items

Field	Type	Description
action	string	Candidate action name.
score	number	Candidate score.
eligible	boolean	Whether the action passed eligibility rules.
reason_codes	array[string]	Reasons behind scoring or blocking.

9. Eligibility vs Scoring

Scoring asks: How attractive or relevant is this action for this customer?

Eligibility asks: Is the system allowed to present this action to this customer?

```

Strong investment characteristics
    |
    v
Investment consent = false
    |
    v
INVESTMENT_INFO = INELIGIBLE
    |
    v
Another eligible action can win

```

In v1, blocked actions receive a score of 0.

10. Reason Codes

Reason codes make the decision explainable and testable. Examples:

```

HAS_MONTHLY_SURPLUS
LOW_SAVINGS_RELATIVE_TO_INCOME
DIGITAL_ENGAGEMENT
NO_MARKETING_CONSENT
NO_INVESTMENT_CONSENT
ALREADY_HAS_MORTGAGE
HIGH_CREDIT_SCORE_BAND
NEGATIVE_MONTHLY_SURPLUS

```

A QA engineer can therefore verify not only the final decision, but also why it was made.

11. Main Source Files

File	Responsibility
src/app.py	FastAPI endpoints.
src/models.py	Pydantic data models.
src/repository.py	Loads and retrieves synthetic customers.
src/decision_engine.py	Eligibility, scoring, ranking, final NBA.
src/synthetic_data.py	Generates 20,000 synthetic customers.
src/demo.py	Runs a sample decision without HTTP.
tests/test_api.py	API-level automated tests.
tests/test_decision_engine.py	Decision-engine automated tests.

12. Start the API

From the project directory, with the **banking** environment active:

```
.\banking\Scripts\Activate.ps1
uvicorn src.app:app --reload
```

Expected server address: `http://127.0.0.1:8000`. Stop the server with **Ctrl+C**.

13. Testing with Swagger UI

Start Uvicorn, open `http://127.0.0.1:8000/docs`, expand an endpoint, select **Try it out**, provide any required parameter, and select **Execute**. For the NBA endpoint, try customer ID C00001.

14. Testing with Postman

```
GET http://127.0.0.1:8000/nba/C00001
```

No authentication, custom headers, or request body are required in v1. Useful checks include: valid customer returns 200; unknown customer returns 404; winner is eligible; ranked actions are ordered by score; and consent restrictions block the correct actions.

15. Current Limitations

v1 is intentionally a training system. It has no real bank data, authentication, authorization, production database, external banking integration, ML model, model registry, audit database, Pega integration, GCP deployment, or production monitoring.

16. Planned v2 - Machine Learning

```
Historical synthetic interactions
|
v
Feature engineering
|
v
Propensity ML model
|
v
P(accept | customer, context, action)
|
v
Eligibility rules
|
v
NBA ranking
|
v
```

Planned additions include Logistic Regression and Gradient Boosting baselines, train/test split, ROC AUC, precision/recall, calibration, model versioning, champion/challenger comparison, and regression tests.

Core principle: ML predicts - decisioning decides - QA verifies.

17. Interview-Level Explanation

"I built a synthetic Next-Best-Action API using FastAPI. The system evaluates multiple candidate actions for each customer, applies eligibility constraints, scores and ranks the eligible actions, returns the winning decision with reason codes, and exposes the result through a REST endpoint. I also created automated tests for decision correctness and API behavior. The first version is rule-based so that decisioning is explicit; the next version introduces ML propensity models behind the same API."

18. Safety and Scope

This project is for training and portfolio use only. Do not use it to make real financial decisions. The customer records are synthetic and the scoring rules are educational examples, not financial advice or production banking logic.